

SEQUELIZE ORM

Arquitectura y Mapeo Objeto-Relacional - Unidad 7

1. El Paradigma ORM: Abstracción de Hardware Relacional

Un **ORM (Object-Relational Mapping)** es una capa de software que actúa como traductor entre el paradigma orientado a objetos de la memoria RAM (Node.js/JavaScript) y el almacenamiento físico estructurado del disco duro (SQL).

Análisis de Costo/Beneficio (Overhead vs Productividad):

- **Ventajas:** Destruye la necesidad de escribir sentencias SQL crudas, previniendo automáticamente vulnerabilidades de inyección SQL (parametrización nativa). Facilita la portabilidad si mañana cambias el motor de BD (de MySQL a PostgreSQL, el código JS no cambia).
- **Desventajas:** Añade un *overhead* (carga adicional) en la CPU y RAM. Para consultas altamente complejas o reportes masivos con múltiples JOINS anidados, el SQL generado por el ORM puede ser ineficiente. Un desarrollador de élite sabe cuándo usar el ORM y cuándo bajar al SQL crudo.

2. Arquitectura de Conexión (Instanciación)

Sequelize requiere un puente físico hacia el motor de la base de datos. Se configura creando una instancia de la clase `Sequelize` con las credenciales de acceso. **Regla estricta:** Jamás quemar credenciales en el código; deben inyectarse mediante el *environment* (Variables de Entorno).

```
const { Sequelize } = require('sequelize');

// El Socket de conexión
const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
```

```

process.env.DB_PASS,
{
  host: process.env.DB_HOST,
  dialect: 'mysql', // o 'postgres', 'sqlite'
  logging: false    // Apagar el log de SQL en producción para
                    // ahorrar I/O
}
);

```

3. Mapeo Físico: Definición de Modelos

Un "Modelo" es el plano arquitectónico en JavaScript que representa una tabla física en el disco duro. Se definen las columnas, los tipos de datos estrictos y las restricciones (constraints) de hardware.

```

const { DataTypes } = require('sequelize');

const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  username: {
    type: DataTypes.STRING(50),
    allowNull: false,
    unique: true
  },
  role: {
    type: DataTypes.ENUM('admin', 'user'),
    defaultValue: 'user'
  }
}, {
  tableName: 'usuarios', // Fuerza el nombre exacto de la tabla
  física
  timestamps: true      // Crea automáticamente createdAt y
  updatedAt
});

```

Sincronización: El comando `sequelize.sync()` ordena al motor compilar los modelos en tablas SQL. Usar `{ force: true }` destruye y recrea la tabla (Peligro de pérdida de datos). Usar `{ alter: true }` modifica las columnas existentes. **En producción, se anula el sync y se utilizan exclusivamente Migraciones.**

4. Punteros y Multiplicidad: Relaciones entre Modelos

La integridad referencial de SQL (Foreign Keys) se modela en Sequelize combinando métodos de asociación. El motor genera automáticamente las sentencias `JOIN` cuando consultas los datos.

Tipo de Relación	Sintaxis de Sequelize	Impacto en el Esquema Físico
1:1 (One-to-One)	<code>User.hasOne(Profile);</code> <code>Profile.belongsTo(User);</code>	Agrega la llave foránea <code>UserId</code> a la tabla <code>Profiles</code> .
1:N (One-to-Many)	<code>Team.hasMany(Player);</code> <code>Player.belongsTo(Team);</code>	Agrega la llave foránea <code>TeamId</code> a la tabla <code>Players</code> . Un equipo tiene múltiples punteros hacia jugadores.
N:M (Many-to-Many)	<code>Student.belongsToMany(Course, {</code> <code> through: 'StudentCourses' });</code> <code>Course.belongsToMany(Student, {</code> <code> through: 'StudentCourses' });</code>	Crea una Tabla Intermedia (Junction Table) física llamada <code>StudentCourses</code> que almacena las llaves primarias de ambas entidades.

5. Ejecución de Consultas (CRUD y Operadores)

Sequelize traduce las funciones JS a sentencias `SELECT, INSERT, UPDATE, DELETE`. El Optimizador Basado en Costos (CBO) de tu BD recibirá el SQL compilado por estas funciones.

```
// CREATE (INSERT)
const user = await User.create({ username: 'root' });

// READ (SELECT) con Filtros CBO y Operadores
const { Op } = require('sequelize');
const activeUsers = await User.findAll({
  where: {
    status: 'active',
    age: { [Op.gte]: 18 } // Mayor o igual (>=)
  },
  include: [Profile], // Realiza un LEFT OUTER JOIN físicamente
  order: [['createdAt', 'DESC']],
});
```

```
    limit: 10                // Paginación a nivel de disco
  });

  // UPDATE (UPDATE)
  await User.update({ role: 'admin' }, { where: { id: 5 } });

  // DELETE (DELETE)
  await User.destroy({ where: { status: 'banned' } });
```

6. Control Transaccional (ACID) y Hooks

Las bases de datos no perdonan errores en operaciones críticas. Si el paso 1 de un bloque falla, el paso 2 no puede guardarse. Sequelize maneja esto pidiendo un bloque de memoria aislado al motor mediante **Transacciones**.

```
// Transferencia bancaria (Regla ACID estricta)
const t = await sequelize.transaction();

try {
  await Cuenta.decrement('saldo', { by: 500, where: { id: 1 },
transaction: t });
  await Cuenta.increment('saldo', { by: 500, where: { id: 2 },
transaction: t });

  await t.commit(); // Flush a disco magnético
} catch (error) {
  await t.rollback(); // Purga de memoria, revierte todo
}
```

Hooks (Gatillos Lógicos): Funciones que se ejecutan automáticamente en el ciclo de vida de una consulta (antes de insertar, después de actualizar). Su uso principal es aplicar *hashes* criptográficos a las contraseñas antes de hacer el *commit* físico en el disco.

7. Control de Versiones del Disco: Migraciones y Seeders (CLI)

El comando `sequelize.sync()` es tóxico para entornos de producción. La arquitectura correcta dicta el uso del **Sequelize CLI** para versionar la estructura de la base de datos como si fuera código fuente.

- **Migraciones:** Archivos generados secuencialmente que le indican al motor cómo mutar el esquema DDL paso a paso. Poseen dos funciones: `up()` (aplica el cambio al hardware) y `down()` (hace rollback).
- **Seeders:** Archivos para inyectar un conjunto de datos iniciales en las tablas vacías (roles de sistema, administrador base, categorías por defecto).

```
// Comandos del motor CLI
npx sequelize-cli init           // Estructura el directorio
(config, models, migrations)
npx sequelize-cli model:generate // Compila el esqueleto de un
modelo y su migración asociada
```

```
npx sequelize-cli db:migrate      // Ejecuta todas las migraciones  
pendientes  
npx sequelize-cli db:migrate:undo // Revierte el último bloque  
migrado
```

8. Integración Estructural con Express

En una arquitectura de élite, la instancia de `sequelize` no se inicializa en los controladores. Se compila en una capa de infraestructura (ej. `config/database.js`) y se importa en el punto de entrada principal del servidor (el `index` o `app.js`). Los controladores simplemente operan sobre los Modelos importados para procesar la I/O.

Documento compilado a partir de los conceptos de la Unidad 7 - Sequelize y Bases de Datos
Relacionales